

Hallazgos de Ingeniería – API Genérica de Integración con ERPs de Proveedores

Proyecto: Aliflow **Preparado por:** Grupo de Ingeniería **Referencia:** Acta de Reunión Aliflow 3 (25-jun-2026), compromiso de investigación asignado al Grupo de Ingeniería **Fecha de este documento:** 26-jul-2026 **Objetivo:** presentar los hallazgos y recomendaciones de la investigación sobre arquitectura de integración, mecanismos de sincronización, y evaluación de ERPs candidatos (Contífico, Alpwin, Alegra, Odoo Community, ERPNext), para la siguiente reunión de equipo.

1. Resumen ejecutivo

- El reto técnico identificado en el acta ("API genérica que soporte múltiples ERPs sin depender de la implementación específica de cada uno") es real y se confirma con la investigación: los ERPs candidatos son heterogéneos – desde sistemas con API REST moderna hasta sistemas sin ninguna API pública.
- **Recomendación de arquitectura:** patrón Adapter (hexagonal) + modelo de datos canónico + outbox/cola de reintentos para manejo de fallos. Ver sección 3.
- **Recomendación de ERP para la fase inicial (presupuesto 0 USD): Odoo Community**, self-hosted. Es la única opción con soporte confirmado y no contradictorio de facturación electrónica SRI Ecuador, API bien documentada, y costo de licencia 0 USD (solo hosting, ~\$5-10 USD/mes).
- **Recomendación de ERP para una fase futura con presupuesto: Contífico** – jugador establecido en el mercado ecuatoriano, API REST real y documentada, pero sin plan de entrada gratuito.
- **Alegra fue evaluado a fondo y se descarta:** no soporta facturación electrónica para Ecuador (evidencia oficial, ver sección 2.3). Su API técnica es de las mejores documentadas que revisamos, pero no resuelve el problema fiscal real del proveedor ecuatoriano.
- **ERPNext se descarta** por no tener localización fiscal de Ecuador confirmada.
- **Alpwin** no tiene API pública – solo aplicable como caso de integración manual/por archivos si un proveedor específico ya lo usa y no puede migrar.
- Adicionalmente, se revisó el flujo funcional documentado ([Flujos-Aliflow-Revision.html](#)) y se detectó una **contradicción entre el Acta y el flujo** respecto al comprobante tributario, además de varios vacíos no cubiertos aún (ver sección 5).

2. Evaluación de ERPs candidatos

2.1 Tabla comparativa

ERP	Costo real	API pública	Facturación SRI Ecuador	Esfuerzo operativo
Alpwin (Syscompsa)	No determinado — requiere contacto directo	✗ No documentada públicamente	Sí es su función core, pero sin API para automatizar	Alto — integración manual/archivos/BD puente
Contífico	Debe negociarse como cliente; sin plan de entrada gratuito ni sandbox público	✓ REST documentada (<code>api.contifico.com</code>), sin webhooks	✓ Sí — es su especialidad, mercado ecuatoriano	Bajo (SaaS)
Alegria	~\$5 USD/mes en países soportados; trial 15 días	✓ REST muy bien documentada (<code>developer.alegra.com</code>), con webhooks parciales (CRUD, no stock)	✗ No soportado — ver 2.3	Muy bajo (SaaS), pero irrelevante por lo anterior
Odoo Community	0 USD licencia (LGPLv3) + ~\$5-10 USD/mes de VPS	✓ 3 protocolos (XML-RPC, JSON-RPC, REST)	✓ Sí — módulo oficial <code>l10n_ec</code> (firma XAdES, ambientes prueba/producción SRI)	Medio-alto — self-hosted, requiere configuración y mantenimiento
ERPNext (Frappe)	0 USD licencia + ~\$5 USD/mes (Frappe Cloud)	✓ REST documentada	✗ No confirmada para Ecuador	Bajo (cloud)

2.2 Contífico — detalle técnico confirmado

- Base URL: `https://api.contifico.com/sistema/api/v1/`
- Formato REST/JSON. Autenticación por API Key en header `AUTHORIZATION: SECRETKEY`, entregada por soporte al darse de alta como cliente.

- Endpoints relevantes: GET/POST /producto/, GET /producto/{id}/stock/, GET/POST /movimiento-inventario/, POST /documento/ (factura electrónica).
- **No hay evidencia de webhooks** — modelo de consulta/push desde Aliflow hacia Contífico, no de notificación inversa.
- No se encontraron rate limits ni costos públicos — probablemente se negocian directamente con soporte.

2.3 Alegra — por qué se descarta (evidencia)

La investigación inicial (basada en resultados de búsqueda) sugería soporte para Ecuador. Una verificación más profunda contra **fuentes oficiales primarias** revirtió esa conclusión:

- El centro de ayuda oficial de Alegra (ayuda.alegra.com/int) lista explícitamente los países soportados: Colombia, México, República Dominicana, Costa Rica, España, Panamá, Perú, Argentina, Venezuela. **Ecuador no aparece.**
- alegra.com/ecuador/ devuelve **404**, mientras países sí soportados tienen landing page propia confirmada (ej. alegra.com/costarica/facturacion/).
- Búsqueda restringida al dominio oficial (`site:alegra.com ecuador`) no arroja páginas de producto para Ecuador.
- Control de calidad: se confirmó que Colombia/Costa Rica/México sí aparecen sin ambigüedad en las mismas fuentes, descartando que la ausencia de Ecuador sea un fallo de búsqueda.
- Las afirmaciones previas de "soporte SRI Ecuador" venían de resúmenes automáticos de búsqueda sin fuente real y de un sitio de reseñas de terceros internamente inconsistente — no de documentación oficial de Alegra.

Lección para el equipo: para decisiones de este peso, verificar siempre contra la fuente primaria oficial (centro de ayuda, landing page del producto), no contra resúmenes generados o reseñas de terceros.

Técnicamente, la API de Alegra sí es la mejor documentada que se revisó (Basic Auth simple, rate limit de 150 req/min, endpoints claros de `items`, `inventory-adjustments`, `invoices`, `contacts`, `credit-notes`) — queda como referencia de "buena práctica de API" aunque no se use para Ecuador.

2.4 Odoo Community — detalle técnico confirmado

- Community Edition es gratis y open-source (LGPLv3); incluye Inventario y Facturación con el mismo motor contable que Enterprise (se pierde automatización avanzada, no la funcionalidad núcleo).
- **Localización Ecuador oficial** (`l10n_ec`): plan de cuentas NEC, validación RUC/cédula, generación y firma XML (XAdES-BES), integración SOAP con el SRI (ambientes prueba y producción), módulo de Punto de Venta, guía de remisión electrónica, reporte ATS. Confirmado en la documentación oficial de Odoo 19.0.
- API: XML-RPC (máxima compatibilidad histórica), JSON-RPC, y REST (desde Odoo 17/19, OpenAPI-compliant). Auth vía base de datos + usuario + contraseña o API key.
- Costo real de "gratis": sin costo de licencia, pero requiere un VPS (~\$5-10 USD/mes) y esfuerzo de configuración/mantenimiento (certificados SRI, actualizaciones, backups).

2.5 Alpwin — detalle confirmado

- Es un sistema de contabilidad/facturación de **Syscompsa S.A.**, aparentemente un ERP contable genérico para pymes, no un POS especializado en alimentos.
- No se encontró documentación pública de API, portal de desarrolladores, ni referencias de integración REST/webhooks.
- Mecanismos de integración esperables: exportación/importación de archivos (CSV/Excel), acceso directo a su base de datos, o conector a medida negociado con Syscompsa.
- **Recomendación:** solo construir un adaptador para Alpwin si un proveedor real de Aliflow ya lo usa y no puede migrar; en ese caso el adaptador probablemente sea basado en archivos, no en llamadas API síncronas.

3. Arquitectura recomendada para la API genérica

3.1 Patrón Adapter + Arquitectura Hexagonal

El núcleo de Aliflow no debe conocer nada de Odoo, Contífico o Alpwin directamente. Se define un **puerto** (interfaz) en el dominio:

```
interface IInventoryProvider {  
    getMenu(tenantId): MenuItem[]  
    updateStock(tenantId, itemId, delta): Result  
    notifySale(tenantId, orderId, items): Result  
    getSyncStatus(tenantId): SyncStatus  
}
```

Cada ERP tiene un **adaptador concreto** (`OdooAdapter`, `ContificoAdapter`, `AlpwinAdapter`) que traduce esta interfaz a las llamadas y formatos propios de cada sistema. Un `ProviderAdapterFactory` resuelve en runtime qué adaptador instanciar según el `tenantId`. Esto aplica directamente el principio de Inversión de Dependencias (SOLID) que exige la rúbrica del proyecto.

3.2 Modelo de datos canónico

Definir un esquema intermedio propio de Aliflow (`CanonicalProduct`, `CanonicalStockEvent`, `CanonicalInvoiceRef`) con solo los campos que el negocio necesita. Cada adaptador mapea su formato nativo → canónico y viceversa, evitando que un cambio de esquema en un ERP externo rompa el modelo de datos interno.

3.3 Estrategias de sincronización

Estrategia	Cuándo usarla	Trade-off
Push en tiempo real	ERPs con API síncrona (Odoo, Contífico)	Requiere manejo de fallos de red inmediato

Estrategia	Cuándo usarla	Trade-off
Polling periódico	ERPs sin webhooks (caso probable de Alpwin)	Introduce latencia — ventana de "vendido antes de confirmar" ya identificada como riesgo
Batch/reconciliación	Red de seguridad diaria, no mecanismo primario	Detecta y corrige divergencias que el push no resolvió

3.4 Manejo de fallos

- **Outbox pattern:** cada venta genera un evento `StockUpdateRequested` en la misma transacción que la orden; un worker lo consume y llama al adaptador correspondiente, con reintentos y backoff exponencial.
- **Idempotencia:** cada evento lleva un `eventId` único para evitar dobles descuentos si se reintenta.
- **Tabla de reconciliación:** eventos `pending` / `failed` visibles para el proveedor en su panel ("última sincronización exitosa"), evitando la "venta huérfana" ya identificada como riesgo en el flujo funcional.

3.5 Multi-tenant

Cada proveedor tiene su propio conjunto de credenciales de ERP, almacenadas cifradas en una tabla `provider_integration_config` separada del dominio — nunca hardcodeadas por adaptador.

3.6 Mapeo a entregables UML del proyecto

- **Diagrama de componentes:** `Aliflow Core` → `Integration Layer` (expone `IInventoryProvider`) → `OdooAdapter` / `ContificoAdapter` / `AlpwinAdapter` → `Retry Worker` / `Reconciliation Store`.
- **Diagrama de despliegue:** nodo backend de Aliflow, nodo/worker de sincronización, y nodos externos representando los servidores de cada ERP (fuera de control de Aliflow, conexión HTTPS).
- **Diagrama de clases:** la interfaz `IInventoryProvider` + implementaciones concretas es un ejemplo directo de Dependency Inversion y Open/Closed (SOLID), relevante para el rubro de diagramas de clases de la rúbrica.

4. Ruta de implementación sugerida

1. **Fase 0 — piloto, presupuesto \$0 (ahora):** levantar Odoo Community self-hosted (el propio equipo de Ingeniería puede hacerlo) y construir el `OdooAdapter` como primera implementación real de `IInventoryProvider`.
2. **Fase 1 — con ingresos del cliente:** evaluar migración/adición de **Contífico** como alternativa más pulida y de marca reconocida en Ecuador, sin tocar el core de Aliflow (solo se agrega `ContificoAdapter`).

3. **Caso especial — proveedor ya usa Alpin u otro ERP sin API:** construir un adaptador basado en archivos/polling de base de datos, aislado detrás del mismo contrato `IInventoryProvider`.

5. Revisión del flujo funcional (Flujos-Aliflow-Revision.html)

Nota sobre los identificadores (`est-2`, `prov-6`, `op-5`, etc.): son los IDs internos que la herramienta HTML `Flujos-Aliflow-Revision.html` le asigna a cada paso, para poder engancharle comentarios ahí mismo. El patrón es `{rol}-{número}`: `est-` = pasos del rol Estudiante, `prov-` = Proveedor, `op-` = Operador, numerados en el orden en que aparecen en esa pestaña. Se usan aquí solo como referencia rápida a un paso específico de ese documento; cada uno se acompaña de su título completo.

5.1 Contradicción entre el Acta y el flujo documentado

El Acta (sección 4, 25-jun-2026) indica: *"Compra del almuerzo... Generación de comprobante **válido tributariamente**."*

El flujo del rol Proveedor (paso `prov-6` — "Re-emisión de comprobante tributario", con fecha de la misma reunión) indica lo contrario: *"Aliflow emite un comprobante de compra... **sin validez tributaria**... el proveedor re-emite la factura válida en su propio sistema."*

Recomendación: aclarar y corregir el texto del acta antes de que quede plasmado así en el documento de especificación de requerimientos — el modelo correcto y ya confirmado es el de **re-emisión por parte del proveedor**, no emisión fiscal directa de Aliflow.

5.2 Decisiones abiertas de alto impacto (ya marcadas en el flujo, priorizadas aquí)

1. **Saldo por proveedor vs. saldo único distribuido internamente** (paso `est-2` — "Recarga de tarjeta virtual") — define el modelo de datos de la wallet y cómo interactúa con la API genérica. Recomendado cerrar antes de construir el diagrama de clases.
2. **Modelo de cobro de Aliflow al proveedor** (comisión/suscripción, paso `prov-5` — "Visualización de métricas y operación") — no bloquea el MVP técnico, pero afecta el modelo de datos si se quiere reflejar en los requerimientos.
3. **Formato del código de retiro** (QR vs. numérico, pasos `est-6` — "Retiro del almuerzo" — y `op-2` — "Recepción del estudiante en el punto de entrega") — bloquea el prototipo de alta fidelidad (mockups) exigido en el entregable.

5.3 Vacíos detectados, no flageados aún en el flujo

1. **Estado de "no retiro"/expiración de orden:** el flujo solo contempla los estados "Comprado" y "Entregado" — no hay manejo de una orden que nunca se retira. Esto deja el diagrama de estados (exigido por la rúbrica) con solo dos estados reales.
2. **Roles múltiples por proveedor:** el paso `prov-1` — "Autenticación y acceso" — menciona "el proveedor o personal autorizado", implicando varios usuarios por tenant, sin que el flujo modele

permisos diferenciados.

3. **Múltiples operadores/puntos de entrega por proveedor:** no está definido si una orden queda amarrada a un punto físico específico.
 4. **Notificaciones al estudiante:** no hay paso sobre avisos de confirmación de compra o fallos de sincronización — si está fuera de alcance de v1, debería decirse explícitamente (como se hizo con devoluciones).
 5. **Riesgo de "sin modo offline" del operador** (paso op-5 — "Manejo de excepciones" —, ya confirmado como decisión de negocio): es un punto único de falla real y debería quedar registrado explícitamente en el registro de riesgos formal del proyecto (entregable 01.g), no solo como callout en el flujo.
-

6. Pendientes para la siguiente reunión

- ☐ Validar con Negocios la corrección de la sección 4 del acta (comprobante tributario).
 - ☐ Decidir modelo de saldo (por proveedor vs. unificado).
 - ☐ Decidir formato del código de retiro (QR/numérico).
 - ☐ Confirmar con Negocios si existe un proveedor real ya identificado y qué ERP usa actualmente (esto valida o descarta la necesidad del adaptador Alpwin).
 - ☐ Aprobar la ruta de implementación por fases (Odoo Community → Contífico) con el equipo y, si aplica, con el cliente.
 - ☐ Definir estado de expiración/no-show para órdenes no retiradas.
 - ☐ Registrar formalmente el riesgo de "sin modo offline" en el documento de gestión de riesgos.
-

Documento preparado por el Grupo de Ingeniería a partir de investigación técnica (documentación oficial de Contífico, Alegra, Odoo, y fuentes oficiales de soporte por país) y revisión del flujo funcional vigente.